



EC-220 Computer System Architecture

Dr. Ahsan Shahzad

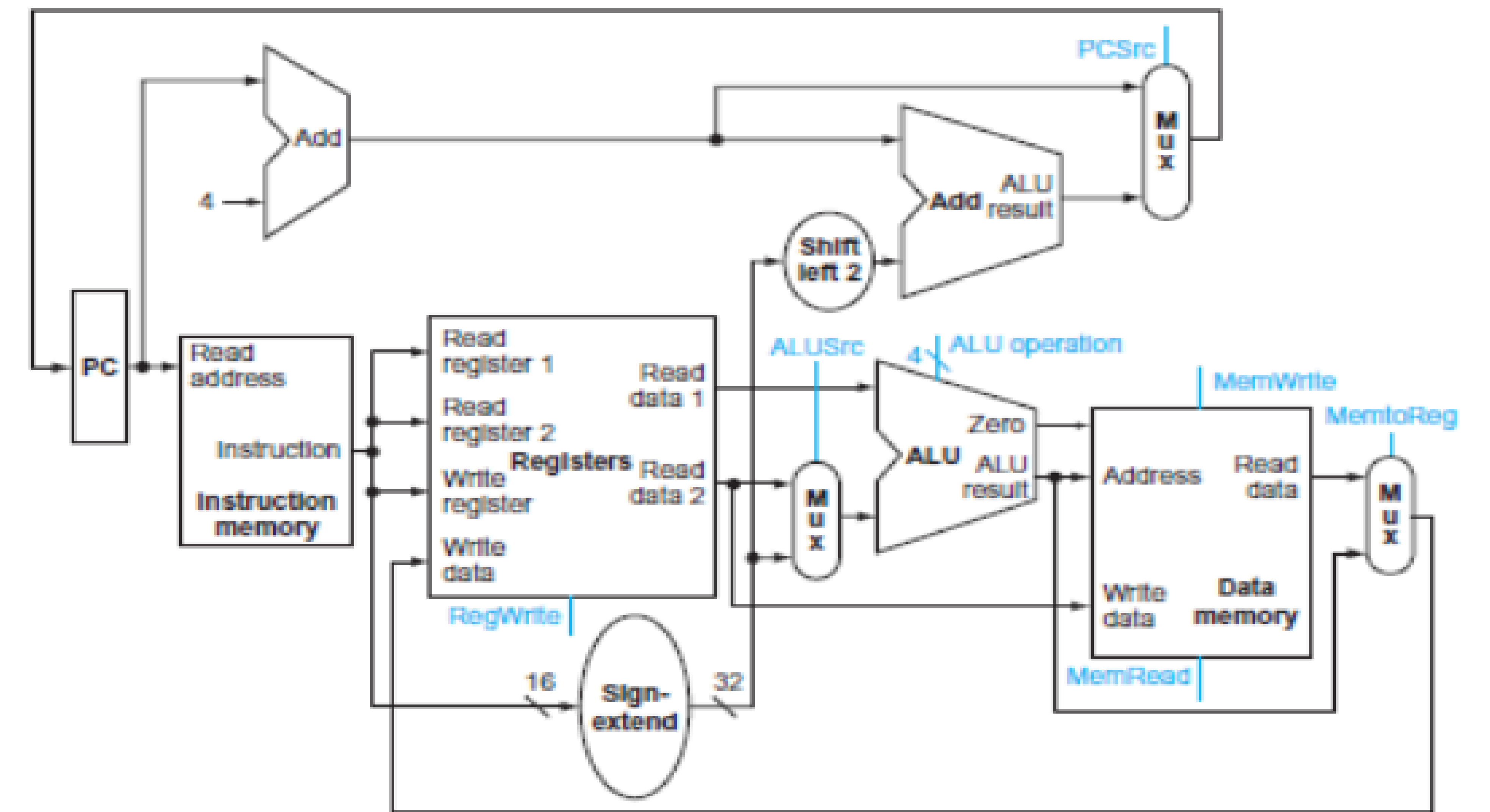
Department of Computer and Software Engineering (DC&SE)

College of Electrical and Mechanical Engineering (CEME)

National University of Science and Technology (NUST)

Week # 12_OLE

Review: DataPath of MIPS Integer Core Subset



4.11 The simple datapath for the core MIPS architecture combines the elements required by d

ALU Control

- **ALU perform one of these 6 functions**
 - NOR will not considered at the moment
- **ALU Control**
 - A small control unit – distributed control
 - Inputs: Function Field [5:0] and ALUOp (2 bits)
 - Output: 4-bit ALU control signals
- **ALUOp**
 - Bits generated by main control
 - Operation to be performed
 - Add (00), subtract (01), using funct. Field (10)

ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract
0111	set on less than
1100	NOR

ALU Control

Instruction opcode	ALUOp	Instruction operation	Funct field	Desired ALU action	ALU control input
LW	00	load word	XXXXXX	add	0010
SW	00	store word	XXXXXX	add	0010
Branch equal	01	branch equal	XXXXXX	subtract	0110
R-type	10	add	100000	add	0010
R-type	10	subtract	100010	subtract	0110
R-type	10	AND	100100	AND	0000
R-type	10	OR	100101	OR	0001
R-type	10	set on less than	101010	set on less than	0111

FIGURE 4.12 How the ALU control bits are set depends on the ALUOp control bits and the different function codes for the R-type instruction. The opcode, listed in the first column, determines the setting of the ALUOp bits. All the encodings are shown in binary. Notice that when the ALUOp code is 00 or 01, the desired ALU action does not depend on the function code field; in this case, we say that we “don’t care” about the value of the function code, and the funct field is shown as XXXXXX. When

ALU Control

ALUOp		Funct field						Operation
ALUOp1	ALUOp0	F5	F4	F3	F2	F1	F0	
0	0	X	X	X	X	X	X	0010
X	1	X	X	X	X	X	X	0110
1	X	X	X	0	0	0	0	0010
1	X	X	X	0	0	1	0	0110
1	X	X	X	0	1	0	0	0000
1	X	X	X	0	1	0	1	0001
1	X	X	X	1	0	1	0	0111

FIGURE 4.13 The truth table for the 4 ALU control bits (called Operation). The inputs are the

Designing the Main Control Unit

- Let's see connection of the instruction fields to the datapath
- Instructions format- Review

Field	0	rs	rt	rd	shamt	funct
Bit positions	31:26	25:21	20:16	15:11	10:6	5:0

a. R-type instruction

Field	35 or 43	rs	rt	address
Bit positions	31:26	25:21	20:16	15:0

b. Load or store instruction

Field	4	rs	rt	address
Bit positions	31:26	25:21	20:16	15:0

c. Branch instruction

FIGURE 4.14 The three instruction classes (R-type, load and store, and branch) use two different instruction formats. The jump instructions use another format, which we will discuss shortly.

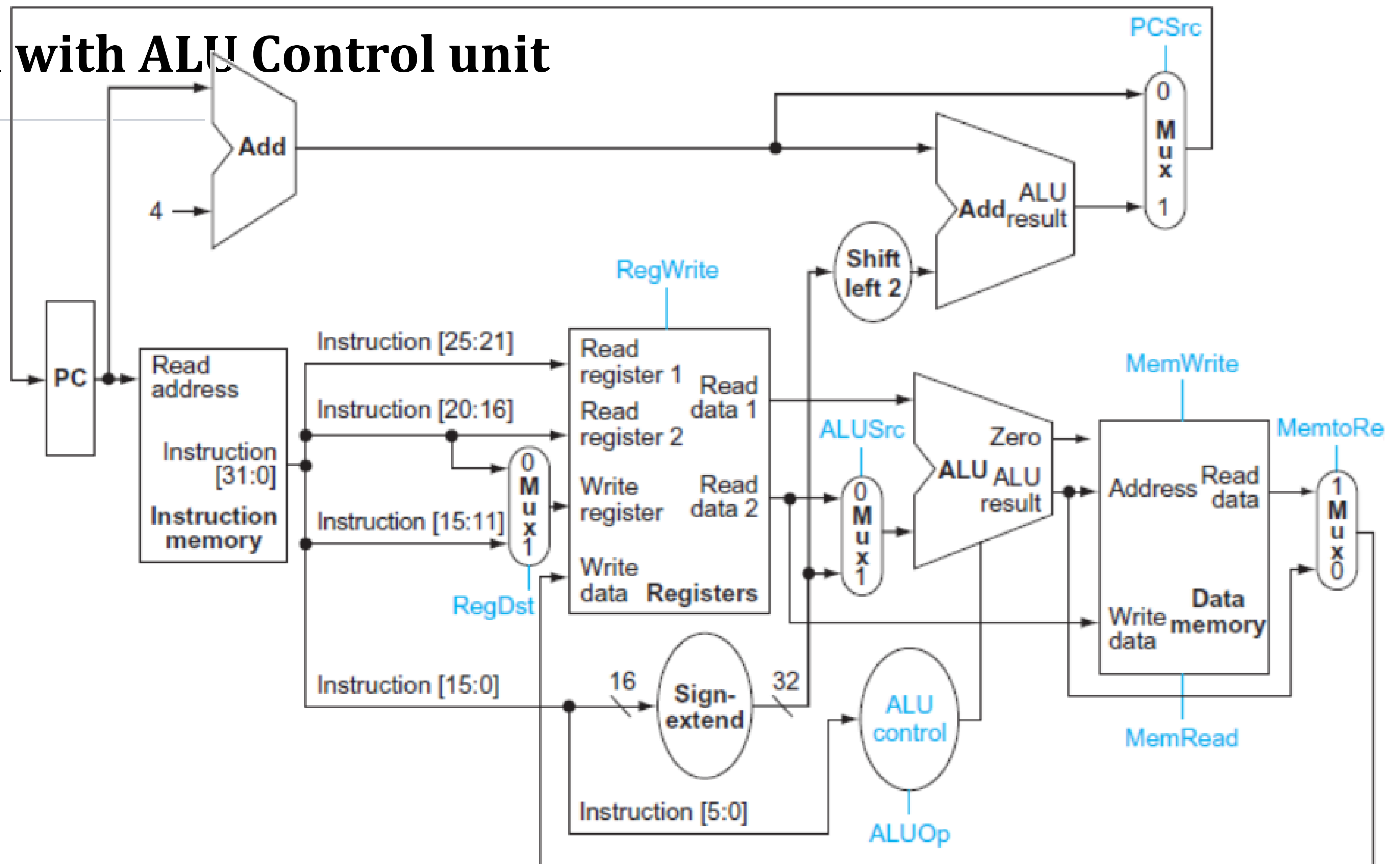
Designing the Main Control Unit

There are several major observations about this instruction format that we will rely on:

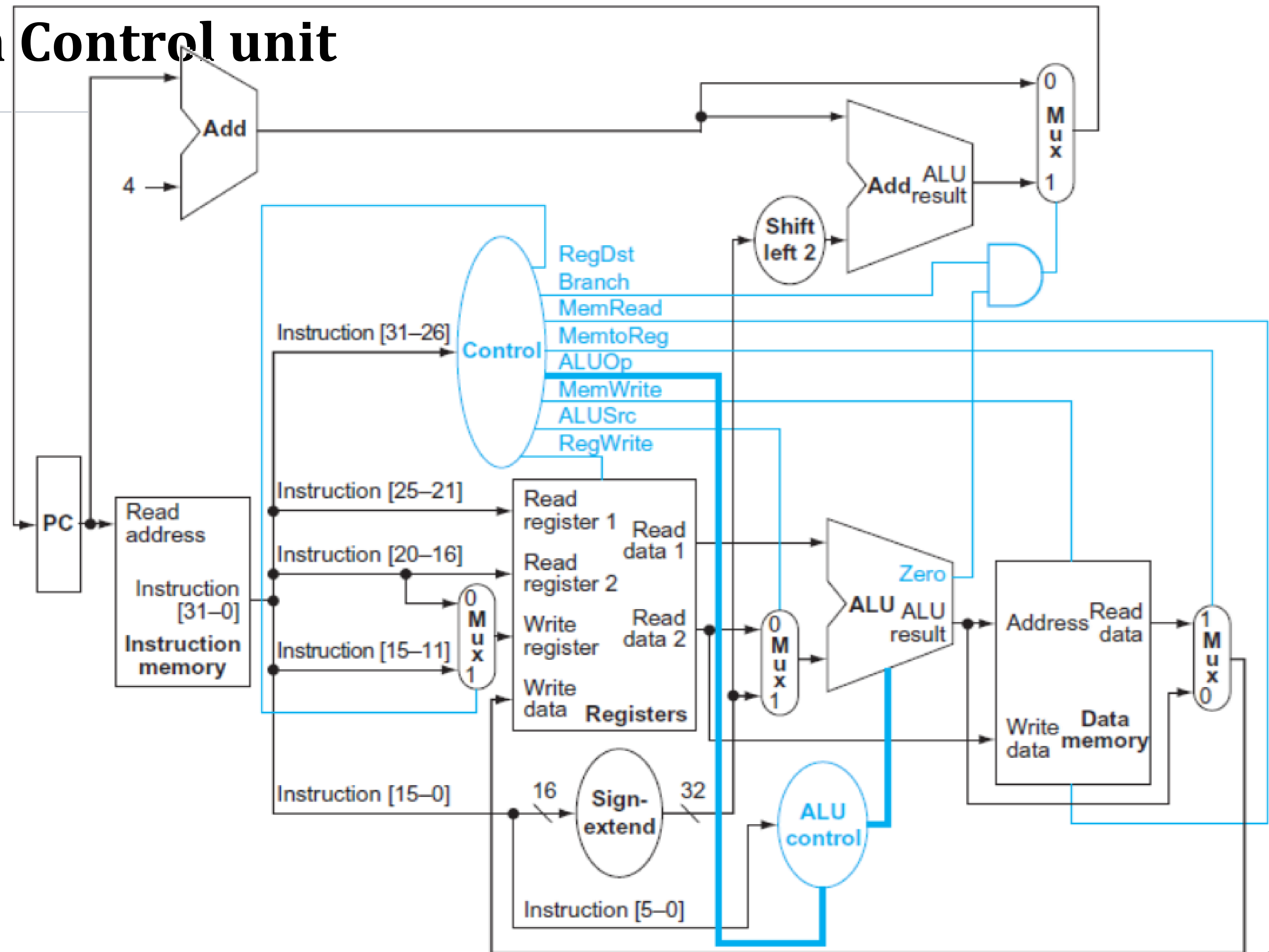
- The op field, which as we saw in Chapter 2 is called the **opcode**, is always contained in bits 31:26. We will refer to this field as Op[5:0].
- The two registers to be read are always specified by the rs and rt fields, at positions 25:21 and 20:16. This is true for the R-type instructions, branch equal, and store.
- The base register for load and store instructions is always in bit positions 25:21 (rs).
- The 16-bit offset for branch equal, load, and store is always in positions 15:0.
- The destination register is in one of two places. For a load it is in bit positions 20:16 (rt), while for an R-type instruction it is in bit positions 15:11 (rd). Thus, we will need to add a multiplexor to select which field of the instruction is used to indicate the register number to be written.

The first design principle from Chapter 2—*simplicity favors regularity*—pays off here in specifying control.

Datapath with ALU Control unit



Datapath with Control unit



Setting of Control Lines

- 9 control signals
 - 3 multiplexors control lines
 - 3 Reg. File W and Mem R/W
 - 1 Branch
 - 2 ALUOp
 - When RegWrite = 0, RegDst and MemtoReg becomes Don't Care

Instruction	RegDst	ALUSrc	Memto-Reg	Reg-Write	Mem-Read	Mem-Write	Branch	ALUOp1	ALUOp0
R-format	1	0	0	1	0	0	0	1	0
lw	0	1	1	1	1	0	0	0	0
sw	X	1	X	0	0	1	0	0	0
beq	X	0	X	0	0	0	1	0	1

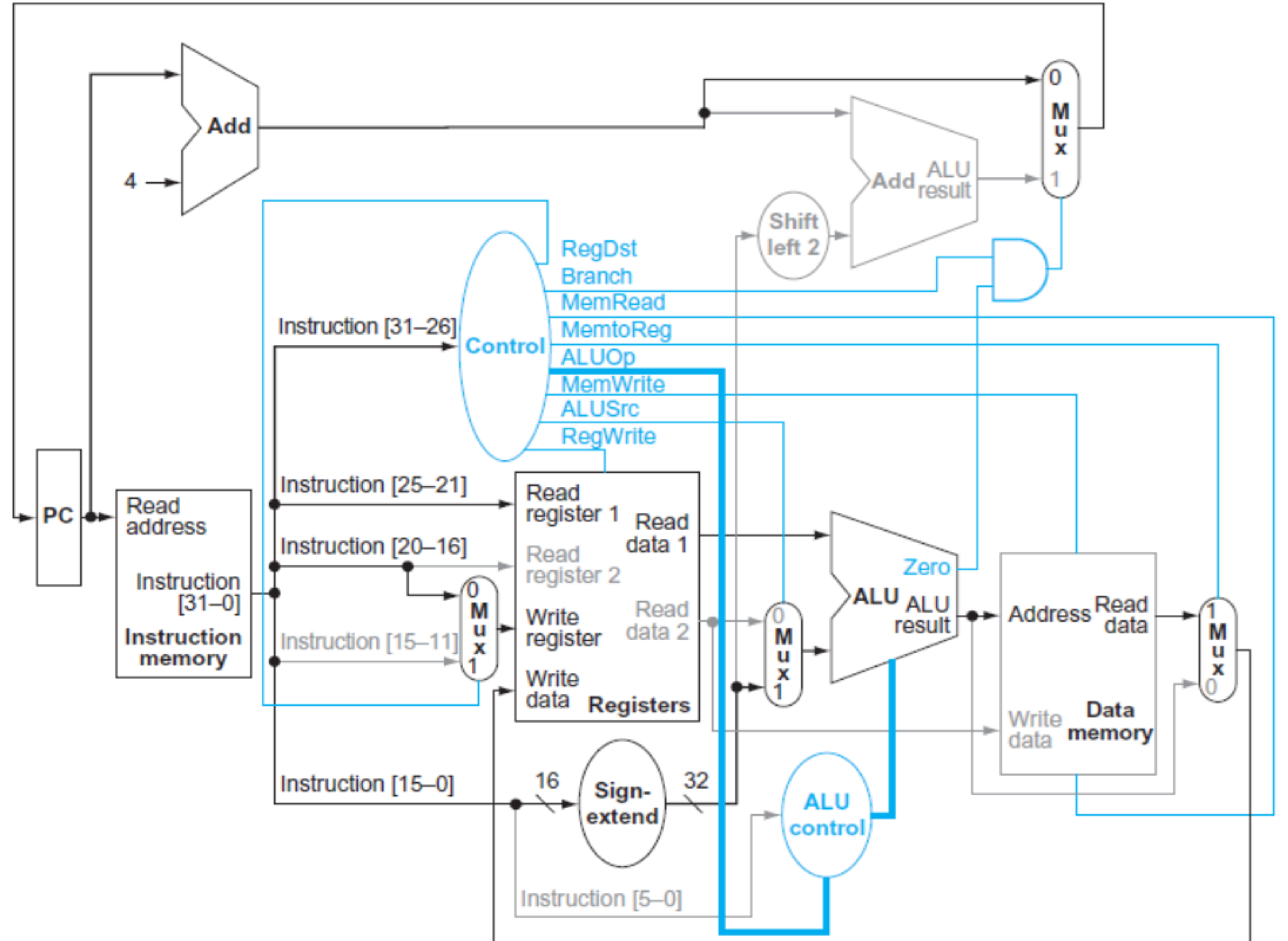
FIGURE 4.18 The setting of the control lines is completely determined by the opcode fields of the instruction. The first

- **Operation of Datapath**



Datapath **Lw (sw)**

- Operation of Datapath



- **Operation of Datapath**



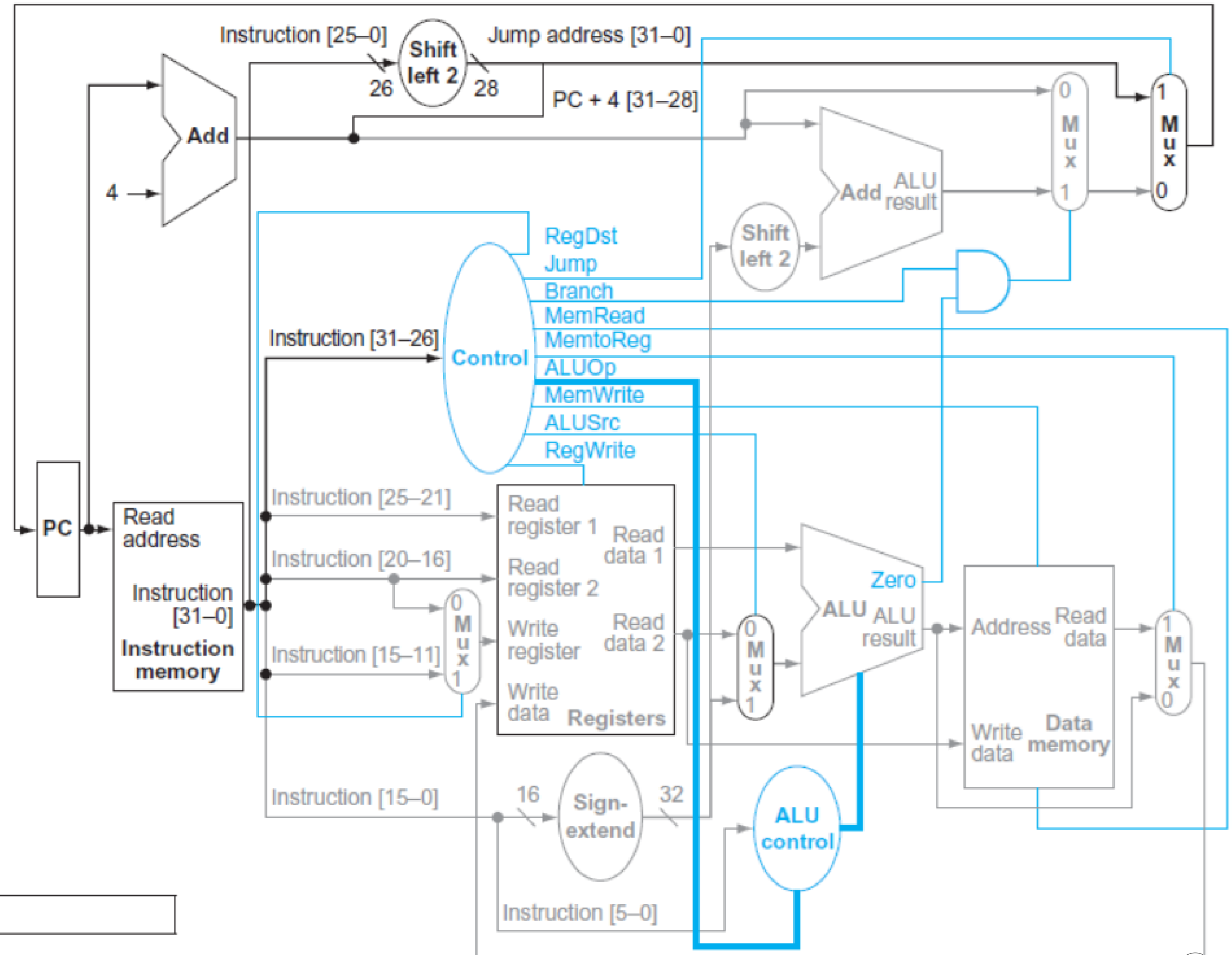
Finalizing Control – Truth Table

Input or output	Signal name	R-format	lw	sw	beq
Inputs	Op5	0	1	1	0
	Op4	0	0	0	0
	Op3	0	0	1	0
	Op2	0	0	0	1
	Op1	0	1	1	0
	Op0	0	1	1	0
Outputs	RegDst	1	0	X	X
	ALUSrc	0	1	1	0
	MemtoReg	0	1	X	X
	RegWrite	1	1	0	0
	MemRead	0	1	0	0
	MemWrite	0	0	1	0
	Branch	0	0	0	1
	ALUOp1	1	0	0	0
	ALUOp0	0	0	0	1

FIGURE 4.22 The control function for the simple single-cycle implementation is completely specified by this truth table. The top half of the table gives the combinations of input

Datapath Jump

- Operation of Datapath



Cons of Single-Cycle Processor

- **Clock Cycle Time depends on the longest instruction**
 - Same Clock cycle length for every instruction
 - $\text{CPI} = 1$, Performance bad due to long Clock cycle.
 - Don't follow design rule: "Making the common case fast"